Project Quality & Coding Standards Report

A source-level review mapping implemented and missing practices across four quality dimensions

Scope	Review of code implementing (or failing to implement) Professional Coding Standards, the 15 Principles of Quality Software, Defensive Programming, and Error / Exception Handling.
Subject	PHP / MySQL crime-mapping web application (includes-based layout, mysqli data layer, Bootstrap/Leaflet front end).
Method	Manual/automated inspection of source files with line-level citation of evidence for each finding.
Page size	Legal — 8.5 in × 14 in.

Status legend used throughout this report:

IMPLEMENTED	PARTIALLY IMPLEMENTED	NOT IMPLEMENTED / MISSING

Table of Contents

Every finding below cites the specific file — and, where determinable, line range — in which the behavior was observed, so entries can be traced directly back to source for verification or patching.

1. Professional Coding Standards and Practices

Practices the codebase follows that align with general professional PHP/MySQL development conventions.

Practice	File(s) / Lines	Status	Evidence / Notes
Consistent layout / includes	**includes/header.php** **includes/footer.php** includes/navbar.php	YES	Shared header/footer/navbar are reused across pages to avoid duplicated UI markup.
Centralized DB connection	**configuration/database.php** L8–L14	YES	Single connection point sets the character set (`$conn->set_charset("utf8")`) instead of repeating connection logic per page.
Centralized session guard	**configuration/session.php** L2–L6	YES	Session start / login-required check is centralized and included on protected pages.
Prepared statements	**authenticate.php** L15–17 **admin/save_crime.php** L60–75 **admin/add_crime.php** L47–51	YES	Parameterized queries with `bind_param` used for most write/read paths.
Password hashing / verification	**generate_hash.php** L8 **authenticate.php** L25	YES	`password_hash()` / `password_verify()` used instead of storing plaintext or weak hashes.
Output escaping	**generate_hash.php** L13	PARTIAL	`htmlspecialchars()` is used in only one location; not applied consistently across all echoed output.

2. Mapping to the 15 Principles of Quality Software

Each principle is mapped to concrete evidence found (or not found) in the codebase.

Principle	Status	File(s)	Evidence / Notes
Correctness	IMPLEMENTED	authenticate.php; admin/save_crime.php	Core authentication and CRUD operations behave correctly for the happy path via prepared statements.
Reliability	PARTIAL	configuration/database.php	Connection failure triggers die(); no retry logic or graceful degradation on failure.
Robustness	PARTIAL	admin/save_crime.php vs admin/add_crime.php	save_crime.php validates required fields; add_crime.php accepts form values without server-side validation.
Security	PARTIAL	admin/hotspot_data.php	Prepared statements + hashing are good practice, but hotspot_data.php uses string-built queries; no CSRF tokens, CSP, or session hardening anywhere.
Maintainability	PARTIAL	includes/*, all page files	Modular includes help, but inconsistent error handling and inline HTML/PHP mixing hurt maintainability.
Readability	IMPLEMENTED	project-wide	Consistent indentation and structure; comments are sparse but code is generally legible.
Efficiency / Performance	MISSING	–	No explicit performance optimizations; acceptable only because datasets are presumed small.
Testability	MISSING	–	No automated tests or test harness exists anywhere in the repository.
Reusability / Modularity	PARTIAL	includes/, configuration/	DB/session centralization is reused, but business logic and presentation are mixed within the same files.
Scalability	MISSING	–	No caching, pagination, or batching addressed anywhere.
Extensibility	PARTIAL	admin/*.php	Modular entry points ease extension, but heavy use of globals/superglobals hinders safe refactors.
Portability	IMPLEMENTED	project-wide	Standard PHP/MySQL (mysqli) code, portable across typical LAMP stacks.
Usability	IMPLEMENTED	includes/header.php, front end	Bootstrap and Leaflet provide a workable, structured UI.

Principle	Status	File(s)	Evidence / Notes
Traceability / Logging	PARTIAL	`authenticate.php (activity_logs insert)`	Only login events are logged; no centralized application/error logging exists.
Documentation	PARTIAL	`README`	A README exists at the project level, but code-level documentation/comments are minimal.

3. Defensive Programming

Implemented defensive-programming practices and their locations.

Practice	File(s) / Lines	Description
Input coercion (type casting)	`admin/save_crime.php; admin/add_crime.php`	`intval()` applied when integers are expected, reducing type-confusion bugs.
Required-field validation	`admin/save_crime.php L23–33`	`empty()` checks on required fields before an INSERT is attempted.
Prepared statements as an injection guard	`authenticate.php; admin/save_crime.php; admin/add_crime.php`	Parameter binding mitigates SQL injection on most write operations.
Session / access guard	`configuration/session.php L2–6`	Forces authentication before protected pages execute any logic.

Gap: defensive checks are applied inconsistently — save_crime.php validates input, but add_crime.php performs no equivalent server-side validation before binding parameters (see Section 5).

4. Error / Exception Handling

4.1 Implemented

Mechanism	File(s) / Lines	Description
Fatal connection check	`configuration/database.php`	Calls `die()` immediately if the DB connection fails.
Prepare-failure check	`admin/save_crime.php L60–65`	Checks `$stmt` after `prepare()` and dies on failure.
Status-flag redirects	`admin/save_crime.php; authenticate.php`	Redirects with `?error=1` / `?success=1` give the user basic feedback.
Activity logging on login	`authenticate.php L51–52`	Successful logins are written to an `activity_logs` table.

4.2 Gaps

Gap	File(s) / Lines	Description
No try / catch or Exceptions	`project-wide`	All error handling is procedural (checks + `die()`); no exception objects are thrown or caught.
No centralized handler / logging	`project-wide`	No `set_error_handler()` or `error_log()` usage; nothing is recorded for later diagnosis.
Raw DB errors shown to users	`admin/add_crime.php L71–73`	`$stmt->error` is echoed directly to the page, leaking internal detail.
Missing input validation on some endpoints	`admin/add_crime.php`	Form handling relies on client-side `required` only; no server-side re-validation.

5. Areas NOT Implementing the Principles (Gap Register)

Consolidated list of code areas and practices that fail to implement the standards reviewed above.

Missing / Non-Compliant Area	File(s) / Lines	Description
Direct SQL string building	`admin/hotspot_data.php L17–22, L33`	Builds WHERE clauses by concatenating `mysqli_real_escape_string`-escaped strings and runs them with raw `mysqli_query()` instead of a prepared statement.
Unvalidated form input	`admin/add_crime.php`	Binds parameters without server-side `empty()` checks or sanitization; relies on client-side `required` only.
No CSRF protection	`all forms`	No CSRF tokens are generated or validated anywhere in the reviewed files.
No centralized error handling / logging	`project-wide`	Errors are handled ad-hoc via `die()`, `echo`, or redirects; no `error_log()` or error-reporting configuration.
No HTTPS enforcement / secure cookie flags	`configuration/session.php`	Session cookie security settings (secure, httponly, SameSite) are not configured.
Raw DB errors echoed to UI	`admin/add_crime.php L71–73`	Displays `$stmt->error` directly, exposing internal error detail to end users.
No input-filtering functions	`project-wide`	`filter_input()` is not used; `$_POST`/`$_GET` are accessed directly and widely.
No unit / integration tests	`repository root`	No test files or test automation of any kind were found.

6. Prioritized Recommendations

- **1.** Replace string-built SQL in `admin/hotspot_data.php` with prepared statements to remove injection risk.
- **2.** Add server-side validation to `admin/add_crime.php` and other form handlers (types, ranges, required fields).
- **3.** Introduce centralized error handling: `set_error_handler()` + `error_log()`; avoid `die()` in production.
- **4.** Stop echoing raw DB errors; log them server-side and show generic, user-friendly messages instead.
- **5.** Add CSRF token generation/validation to all forms.
- **6.** Harden sessions: `session.cookie_secure`, `session.cookie_httponly`, regenerate session ID on login.
- **7.** Adopt `filter_input()` / stricter sanitization, and escape all output consistently.
- **8.** Add basic automated tests for authentication and core CRUD flows.
- **9.** Consider a lightweight data-access abstraction layer to centralize prepared-statement usage and error handling.

7. Summary Checklist

IMPLEMENTED	Prepared statements • password hashing • session guard • modular includes • DB charset configuration.
PARTIAL	Input validation • error/status feedback • defensive input handling (present in some endpoints, absent in others).
MISSING	CSRF protection • centralized error/exception handling • secure session cookie configuration • automated tests • consistent output sanitization.

Appendix A — Code Location Index (File → Line Reference)

Consolidated index of every file/line reference cited in this report, for quick navigation back to source.

File	Line(s)	What's There
configuration/database.php	L8–L14	DB connection setup and charset configuration; die() on connect failure.
configuration/session.php	L2–L6	Session start / centralized login guard for protected pages.
authenticate.php	L3	Session start at top of script.
authenticate.php	L15–L17	Prepared statement setup for login lookup.
authenticate.php	L25	password_verify() check.
authenticate.php	L34–L37	Result handling after authentication attempt.
authenticate.php	L39–L43	Redirect on authentication failure.
authenticate.php	L51–L52	activity_logs insert on successful login.
generate_hash.php	L8	password_hash() generation.
generate_hash.php	L13	htmlspecialchars() output escaping.
admin/save_crime.php	L9–L17	Input coercion (intval) and variable assignment.
admin/save_crime.php	L23–L33	Required-field (empty()) validation.
admin/save_crime.php	L60–L65	Prepare + failure check (die() on prepare failure).
admin/save_crime.php	L67–L75	bind_param() and execute().
admin/save_crime.php	L81–L89	Redirect based on query result.
admin/add_crime.php	L47–L51	Prepare/bind for insert.
admin/add_crime.php	L71–L73	Raw $stmt->error echoed to UI (gap).
admin/add_crime.php	L148	Population query for barangay select list.
admin/hotspot_data.php	L6	Response/header and output type setup.
admin/hotspot_data.php	L17–L22	String-built WHERE clause using mysqli_real_escape_string (gap).
admin/hotspot_data.php	L33	Raw mysqli_query() execution (gap).
includes/header.php	L1–L14	Shared page header / layout skeleton.
includes/footer.php	L1–L6	Shared page footer / closing layout.

This appendix mirrors the “Code locations” section of the underlying review notes (QUALITY_REPORT.md) so that every citation used in Sections 1–5 can be traced to an exact file and line range in one place.